Application web de gestion de tâches

Projet professionnel : TaskPilot

Nom : Boisson Yann

Formation : Développeur Web et Web Mobile full-stack

Centre de formation : Centre Européen de Formation

Année : 2025/2026

Sommaire

Table des matières

Sommaire 2
1 Présentation du projet 4
2 Contexte du projet 5
2.1 — Présentation d'un contexte d'entreprise fictif 5
3 Expression du besoin et cahier des charges 5
4 Contraintes du projet 6
4.1 — Responsive design 7
4.2 — Sécurité 7
4.3 — Accessibilité et ergonomie 7
4.4 — Protection des données 8
5 Livrables attendus 9
6 Environnement humain et technique 10
7 Architecture générale de l'application 10
8 Développement de la partie front-end 11
8.1 — Choix réalisés 12
8.2 — Maquettes 12
8.3 — Présentation des composants 21
8.4 Extrait d'interface statique 22
8.5 Fonctionnalités principales 23
9 — Développement de la partie back-end 25
9.1 — Choix réalisés 25
9.2 — Extrait de composant métier 26
9.3 .— Extrait d'accès aux données 27
10 — Base de données : MCD, MLD et création 28
10.1 — MCD 28

10.2 — MLD 28
10.3 — Création SQL 29
11 — Sécurité de l'application 31
11.1 — Présentation des éléments de sécurité 32
11.2 — Jeu d'essai de sécurité 33
12 — Jeu d'essai 33
13 — Tests réalisés 33
14 Veille sur les vulnérabilités de sécurité 37
15 Conclusion 37
16 Annexes 38
16.1 — Arborescence du projet 38
16.2 — README du projet 38
16.3 — Captures supplémentaires 44
16.4 — Extraits de code complémentaires 46
16.5 — Requêtes SQL utilisées 49

1 Présentation du projet

Le projet **TaskPilot** consiste à développer une application web permettant à un utilisateur de gérer ses tâches personnelles à travers une interface moderne de type **tableau Kanban**.

L'objectif principal est de proposer une solution simple et intuitive pour organiser le travail quotidien, suivre l'avancement des tâches et améliorer la productivité.

Cette application permet à un utilisateur authentifié de **créer**, **consulter**, **déplacer** et **supprimer** ses tâches dans un tableau organisé en colonnes.

Le projet a été réalisé dans le cadre de la formation Développeur Web et Web Mobile full-stack, avec pour objectif de mettre en pratique les compétences liées au développement **front-end**, **back-end**, à la gestion d'une base de données relationnelle ainsi qu'à la sécurisation d'une application web.

—

2 Contexte du projet

Dans de nombreuses situations professionnelles, la gestion des tâches constitue un enjeu important pour l'organisation du travail. Une mauvaise visibilité sur les actions à réaliser peut entraîner des oublis, des retards ou une perte d'efficacité.

Le projet TaskPilot répond à ce besoin en proposant une application web de gestion de tâches personnelles. L'idée est de fournir à l'utilisateur un outil simple d'utilisation, visuel et accessible depuis un navigateur, afin de centraliser ses tâches dans une interface claire.

Le projet s'inscrit dans une logique de digitalisation de l'organisation personnelle et professionnelle. Le choix d'un tableau de type Kanban permet une visualisation immédiate de l'état d'avancement des tâches, ce qui améliore l'ergonomie et la compréhension de l'outil.

L'application a été conçue comme un projet full-stack complet, avec un front-end dynamique en React, un back-end en Node.js / Express et une base de données MySQL.

2.1 — Présentation d'un contexte d'entreprise fictif

Dans le cadre de ce dossier, on peut situer le projet dans une petite structure souhaitant améliorer l'organisation des tâches internes de ses collaborateurs. L'objectif est de disposer d'un outil léger, rapide à prendre en main et accessible depuis un navigateur, sans installation complexe côté utilisateur.

L'entreprise souhaite permettre à chaque collaborateur de gérer ses propres tâches, suivre leur état d'avancement et disposer d'un tableau clair distinguant les tâches à faire, en cours et terminées.

3 Expression du besoin et cahier des charges

Le besoin fonctionnel identifié est la mise à disposition d'une application web permettant la gestion simple et sécurisée de tâches personnelles.

—

Le cahier des charges fonctionnel prévoit les éléments suivants :

- **création** d'un compte utilisateur ;
- **connexion** sécurisée à l'application ;
- **ajout** d'une tâche avec titre et description ;
- **affichage** des tâches dans un tableau Kanban ;
- **modification** du statut d'une tâche par glisser-déposer ;
- **suppression** d'une tâche ;
- **recherche** et filtrage des tâches ;
- **affichage** de statistiques simples sur le tableau de bord.

Le cahier des charges technique prévoit :

- une application front-end développée avec React ;
- une API REST développée avec Node.js et Express ;
- une base de données MySQL ;
- une authentification sécurisée avec JWT ;
- un design responsive et une interface moderne.

4 Contraintes du projet

La réalisation du projet a dû prendre en compte plusieurs contraintes techniques et fonctionnelles.

Tout d'abord, l'application devait être **responsive**, c'est-à-dire utilisable sur différents formats d'écran, notamment ordinateur et mobile, avec une adaptation correcte de la mise en page.

—

Ensuite, des contraintes de sécurité ont été prises en compte. Les mots de passe des utilisateurs ne doivent pas être stockés en clair dans la base de données. De plus, les routes d'accès aux tâches doivent être protégées afin qu'un utilisateur ne puisse consulter que ses propres données.

Des contraintes liées à la qualité globale de l'application ont également été considérées : lisibilité de l'interface, ergonomie, structure claire du code, séparation front-end / back-end et facilité de maintenance.

Le projet tient aussi compte, de manière partielle, de notions importantes comme **l'accessibilité**, le **SEO côté front-end** et le respect des bonnes pratiques de **protection des données personnelles.**

4.1 — Responsive design

L'interface a été pensée pour s'adapter aux différentes tailles d'écran grâce à l'utilisation de **media queries CSS**. Les blocs principaux, les formulaires et le tableau s'ajustent pour conserver une bonne lisibilité.

4.2 — Sécurité

La sécurité repose sur le chiffrement des mots de passe avec bcrypt, l'utilisation d'un **token JWT** pour l'authentification et un **middleware** permettant de protéger les routes API liées aux tâches.

4.3 — Accessibilité et ergonomie

Une attention a été portée à la clarté des libellés, à la simplicité de navigation et à la lisibilité générale de l'interface. L'organisation en colonnes et l'utilisation de boutons clairement identifiés facilitent l'usage.

4.4 — Protection des données

Chaque utilisateur accède uniquement à ses propres tâches. Cette séparation des données participe au respect de la confidentialité et s'inscrit dans une logique de protection des données personnelles.

5 Livrables attendus

Les livrables attendus dans le cadre du projet sont les suivants :

- une **application front-end** fonctionnelle ;
- une **API back-end** sécurisée ;
- une **base de données MySQL** structurée ;
- un **dépôt GitHub** contenant le code source ;
- un dossier projet détaillant la démarche de réalisation ; - un support de présentation pour la soutenance.

Ces livrables permettent de démontrer la maîtrise des différentes compétences visées par la formation.

6 Environnement humain et technique

Le projet a été réalisé individuellement dans le cadre de la formation Développeur Web et Web Mobile full-stack.

L'environnement technique utilisé est le suivant :

- Visual Studio Code pour le développement ;

- **Node.js** et npm pour la gestion des dépendances ;

- **React** pour l'interface utilisateur ;

- **Express** pour le serveur API ;

- **MySQL** et **phpMyAdmin** pour la base de données ;

- **Git** et **GitHub** pour le versionnement du code ; - Word pour la rédaction du dossier projet.

L'environnement de test a été principalement un environnement local, avec exécution du front-end via Vite et du back-end via Node.js.

7 Architecture générale de l'application

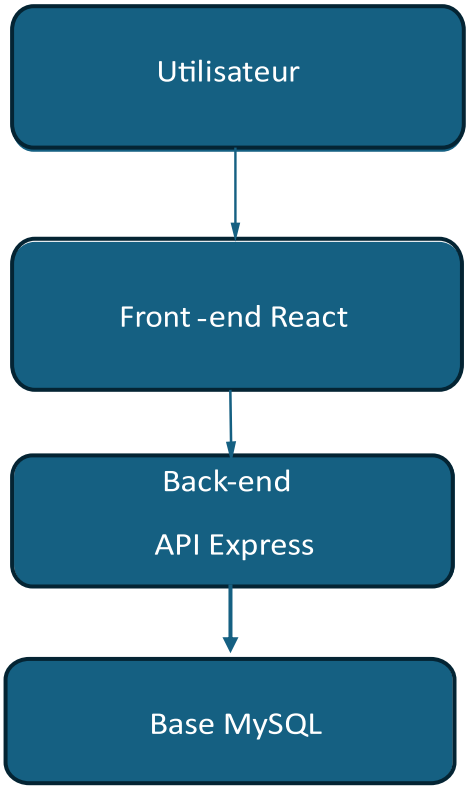
L'application TaskPilot repose sur une architecture full-stack séparée en trois couches principales.

La première couche est le **front-end**, développé avec **React**. Il gère l'affichage des pages, les interactions utilisateur, la navigation et la logique visuelle du tableau Kanban.

La deuxième couche est le **back-end**, développé avec **Node.js** et **Express**. Il expose des routes **API** permettant de gérer l'authentification et les opérations sur les tâches.

La troisième couche est la base de données **MySQL**. Elle stocke les utilisateurs ainsi que les tâches liées à chaque utilisateur.

Le front-end communique avec le back-end via des requêtes HTTP envoyées grâce à Axios. Le back-end communique ensuite avec MySQL pour lire ou écrire les données.



L'architecture de l'application TaskPilot repose sur une séparation claire entre l'interface utilisateur, le serveur et la base de données.

L'utilisateur interagit avec le **front-end** développé en **React**. Le **front-end** envoie **des requêtes HTTP** vers **l'API REST** développée avec **Node.js** et **Express**.

Le **back-end** traite **les requêtes**, applique les règles métier, **vérifie l'authentification** et communique avec la **base de données MySQL**.

La base de données stocke les utilisateurs et les tâches. Elle n'est jamais appelée directement par le front-end.

8 Développement de la partie front-end

La partie front-end de l'application a été développée avec React. Le choix de cette bibliothèque s'explique par sa capacité à construire des interfaces modernes sous forme de composants réutilisables.

8.1 — Choix réalisés

Le choix de **React** permet d'avoir une interface **dynamique**, **rapide** et **structurée**. L'approche par composants facilite la maintenance du projet et la réutilisation des éléments d'interface.

Le choix d'un **tableau Kanban** améliore **l'ergonomie** en permettant à l'utilisateur de visualiser immédiatement l'état d'avancement de ses tâches.

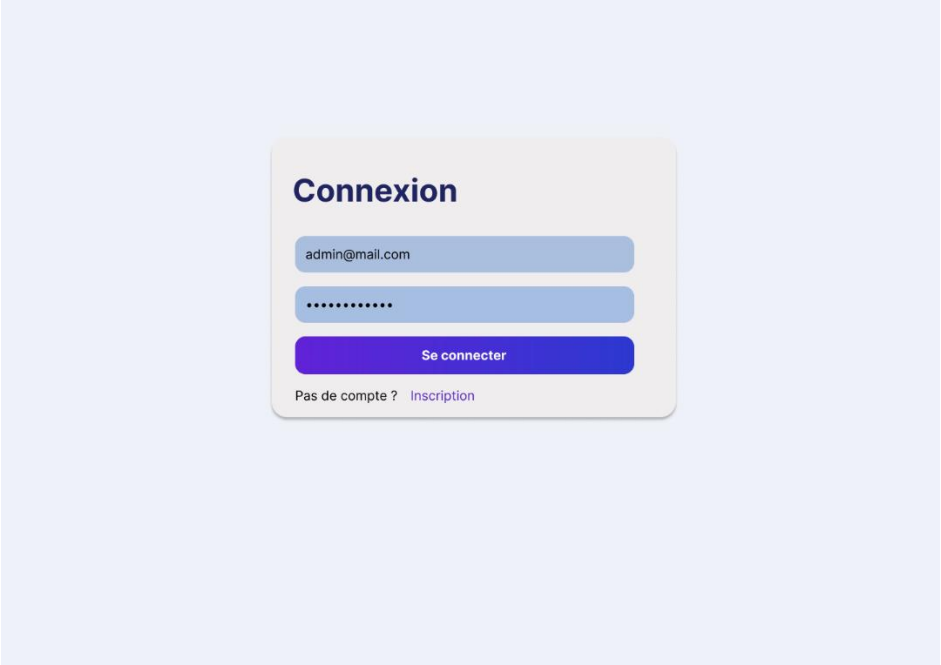
8.2 — Maquettes

Une phase de réflexion visuelle a été menée avant la finalisation de l'interface. L'objectif était d'obtenir une interface claire, simple et inspirée des outils de gestion de tâches existants.

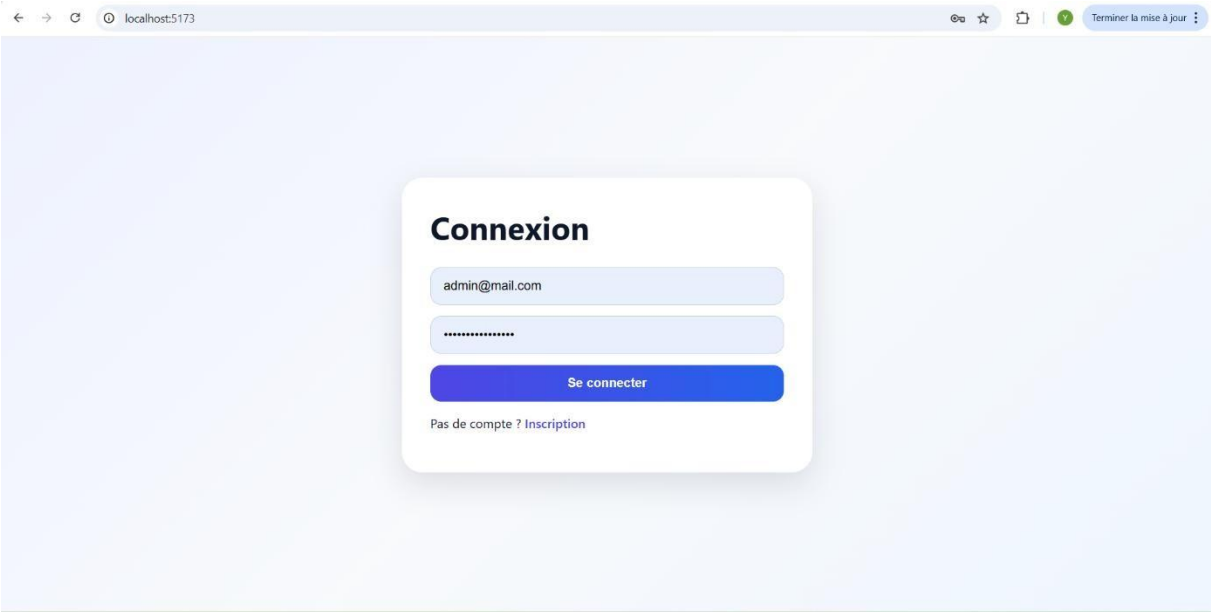
L'interface retenue comporte :

- une page de connexion ;
- une page d'inscription ;
- un tableau de bord central ;
- un tableau Kanban en trois colonnes.

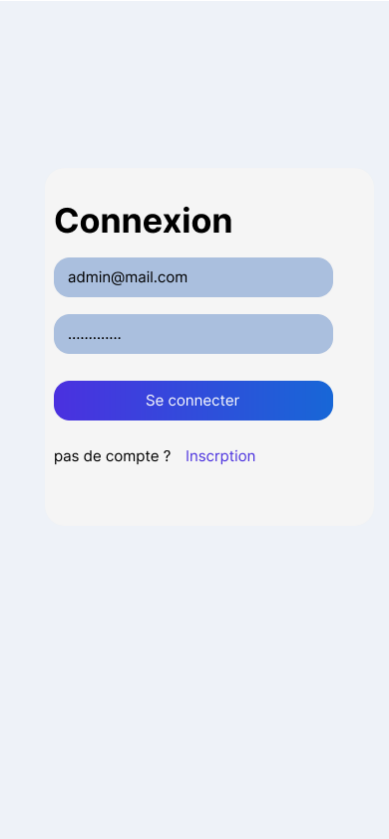
Les choix visuels ont été orientés vers la lisibilité, l'ergonomie et la simplicité d'utilisation.



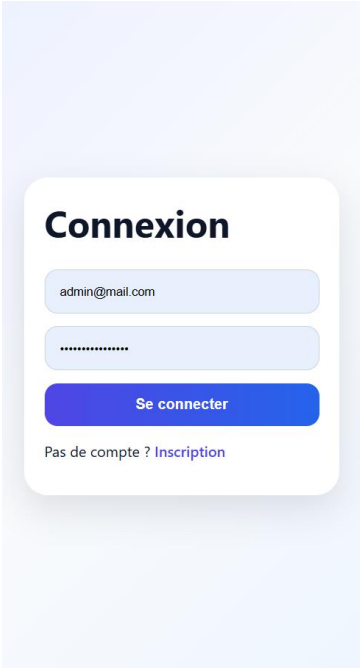
Maquette figma desktop de la page Connexion de l'application TaskPilot



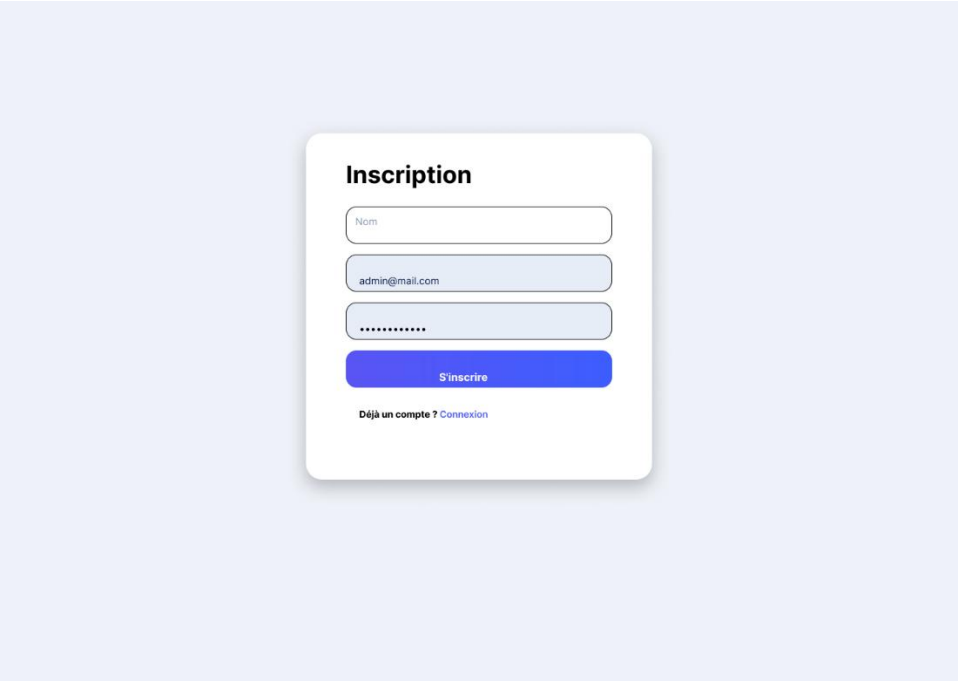
Rendu final de la Page de connexion de l'application taskpilot desktop



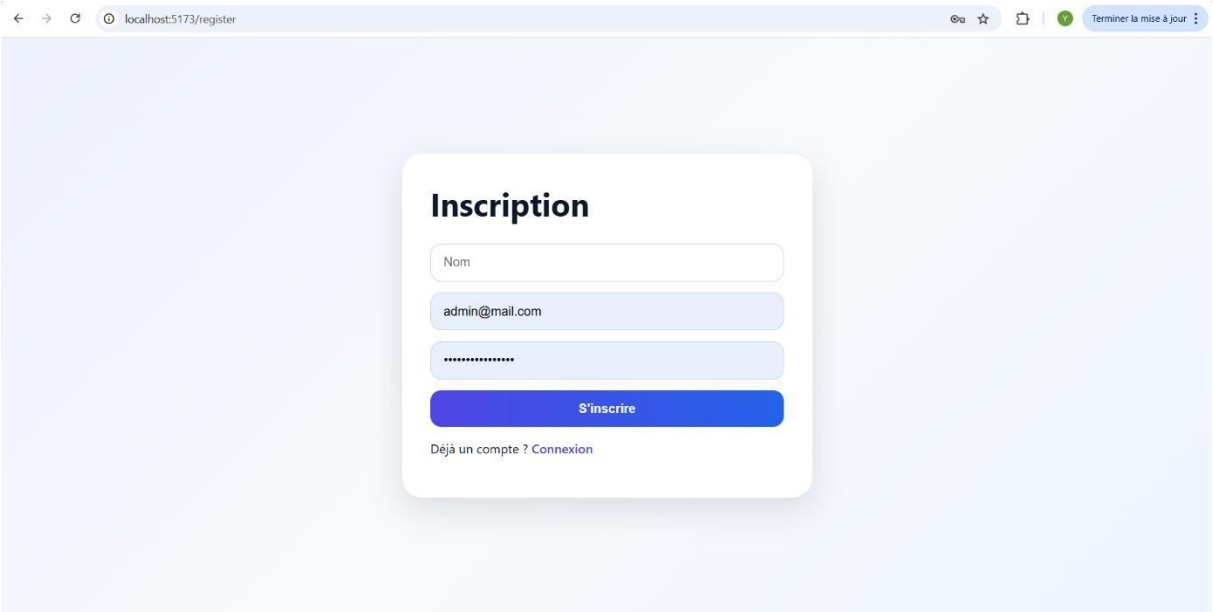
Maquette figma mobile de la page Connexion de l'application TaskPilot



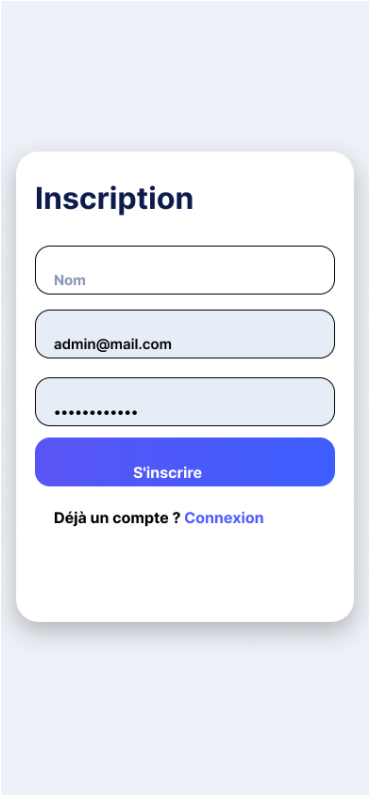
Rendu final de la Page de connexion de l'application taskpilot mobile



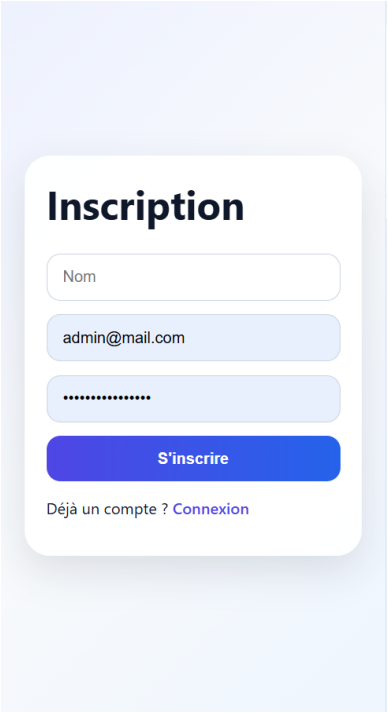
Maquette figma desktop de la page d'inscription de l'application TaskPilot



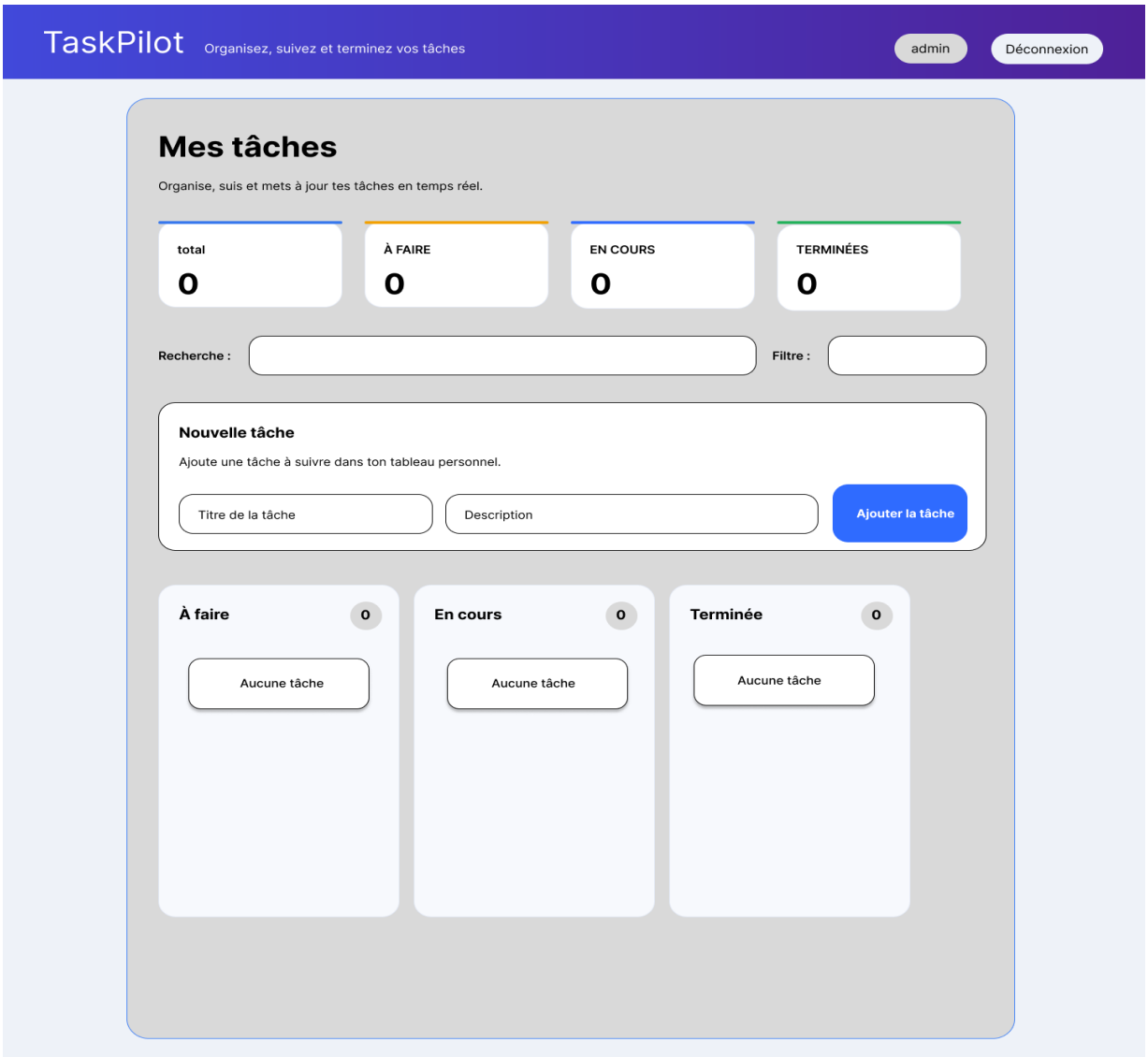
Rendu final de la Page d'inscrition de l'application taskpilot desktop



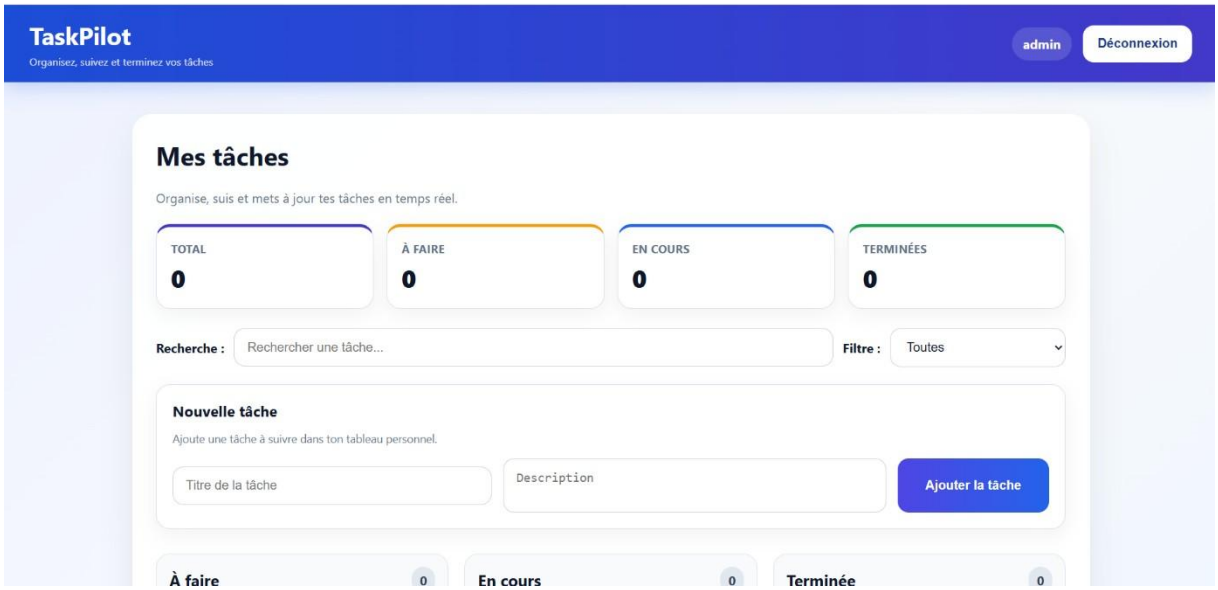
Maquette figma mobile de la page d'inscription de l'application TaskPilot



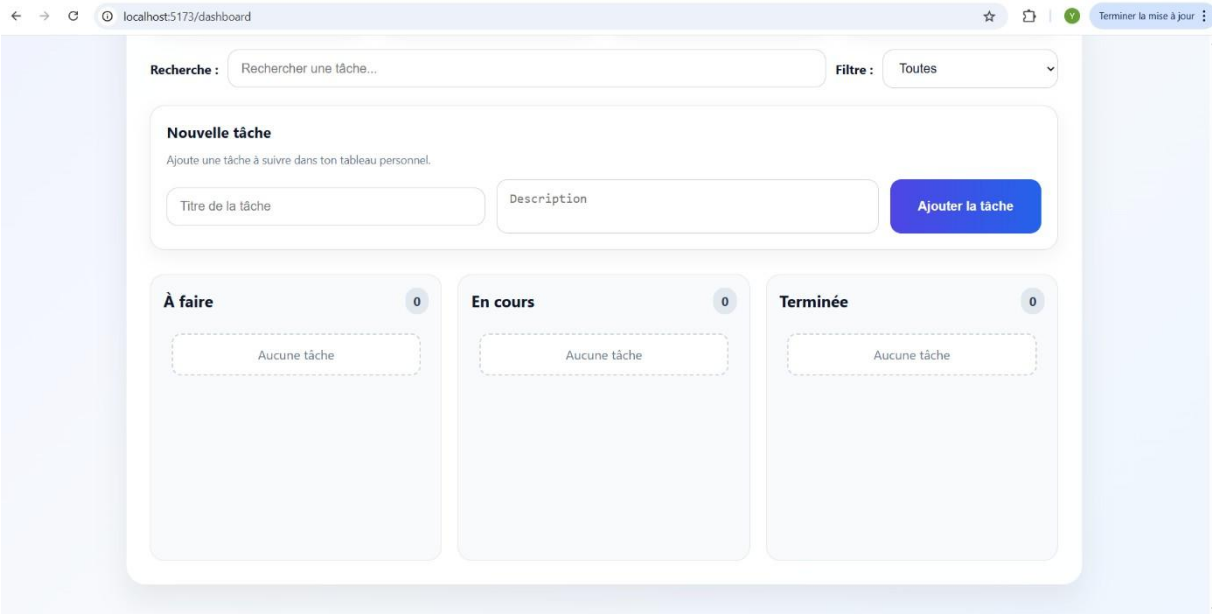
Rendu final de la Page d'inscrition de l'application taskpilot mobile



Maquette figma de la page dashboard de l'application Taskpilot

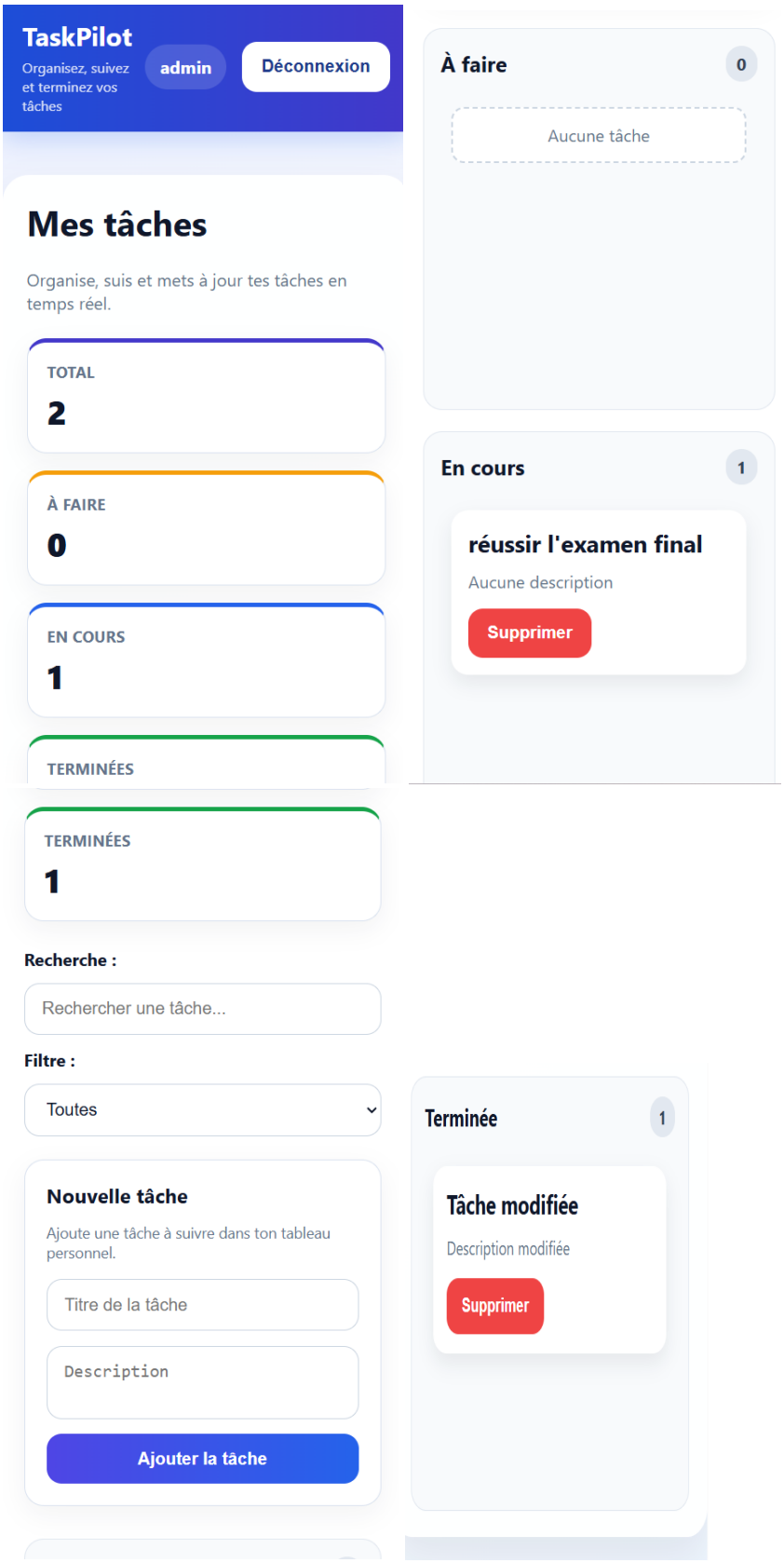


Rendu final de la Page dashboard de l'application taskpilot desktop



Rendu final de la Page dashboard de l'application taskpilot desktop

Maquette figma mobile de la page dashboard de l'application Taskpilot



Rendu final de la Page dashboard de l'application taskpilot mobile

8.3 — Présentation des composants

Le **front-end** de l'application **TaskPilot** est organisé en plusieurs composants **React**.
Cette approche permet de séparer les **responsabilités**, de rendre le code plus **lisible** et de **faciliter la maintenance** du projet.

Les principaux composants utilisés sont les suivants :

- **Login.jsx** : ce composant gère la connexion de l'utilisateur à l'application. Il récupère les identifiants saisis, envoie la requête de connexion au back-end et stocke le token JWT après authentification.

- **Register.jsx** : ce composant permet à un nouvel utilisateur de créer un compte. Il récupère le nom, l'email et le mot de passe, puis transmet ces informations au serveur pour enregistrement.

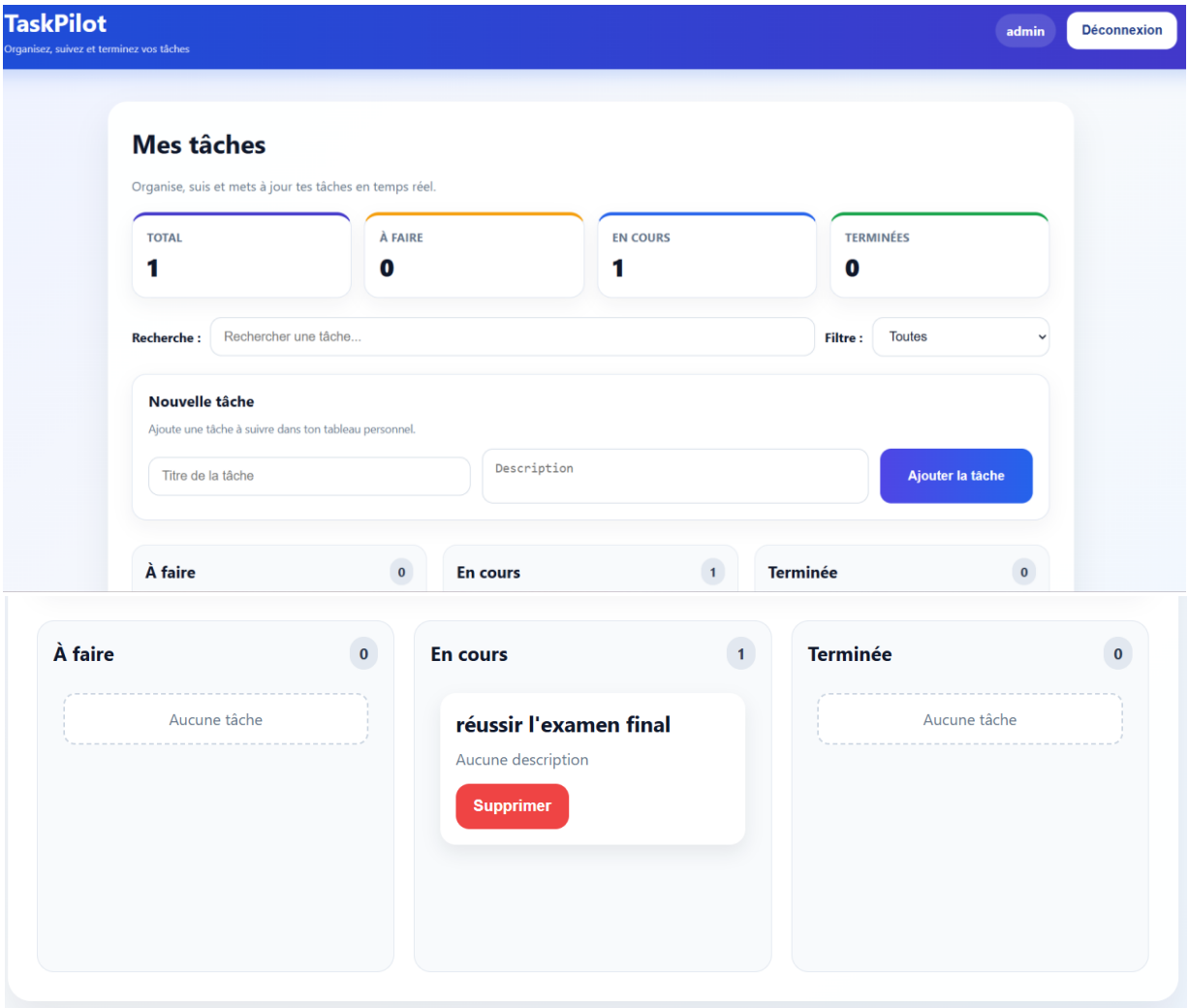
- **Dashboard.jsx** : il s'agit du composant principal de l'application. Il affiche les statistiques, les filtres, le formulaire d'ajout et les différentes colonnes du tableau Kanban.

- **Navbar.jsx** : ce composant affiche l'en-tête de l'application, le nom de l'utilisateur connecté et le bouton de déconnexion.

- **TaskForm.jsx** : ce composant contient le formulaire d'ajout de tâche. Il permet à l'utilisateur de saisir un titre et une description avant validation.

- **TaskList.jsx** : ce composant affiche les tâches dans les colonnes du tableau Kanban. Il gère également certaines interactions comme le déplacement d'une tâche et sa suppression.

Cette organisation en composants rend l'interface plus modulaire. Chaque élément de l'application possède un rôle précis, ce qui facilite les évolutions futures et la réutilisation du code.



8.4 Extrait d'interface statique

```
{/* Formulaire d'ajout de tâche */}
<form onSubmit={handleSubmit} className="task-form">
    <input
        type="text"
        placeholder="Titre de la tâche"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
    />

    <textarea
        placeholder="Description"
        value={description}
        onChange={(e) => setDescription(e.target.value)}
    />

    <button className="primary-btn" type="submit">
        Ajouter la tâche
    </button>
</form>
```

Extrait du formulaire d'ajout de tâche dans le composant TaskForm.jsx.

L'extrait présenté ci-dessus correspond au formulaire d'ajout de tâche du composant TaskForm.jsx.

Cette partie de l'interface permet à l'utilisateur de saisir les informations nécessaires à la création d'une nouvelle tâche.
Le formulaire contient un champ de titre, un champ de description et un bouton de validation.

Cet extrait illustre la **structure statique** de l'interface utilisateur.
Il montre comment les éléments visuels sont organisés dans **React** avant même le traitement dynamique des données.

Le choix d'une interface simple et épurée permet de faciliter la prise en main de l'application.
L'utilisateur comprend immédiatement l'action attendue et peut ajouter une tâche sans difficulté.

8.5 Fonctionnalités principales

L'application **TaskPilot** propose plusieurs fonctionnalités principales permettant à l'utilisateur de gérer efficacement ses tâches.

- **Création de tâche (Create):**
L'utilisateur peut ajouter une nouvelle tâche via un formulaire comprenant un titre et une description. La tâche apparaît immédiatement dans le tableau.

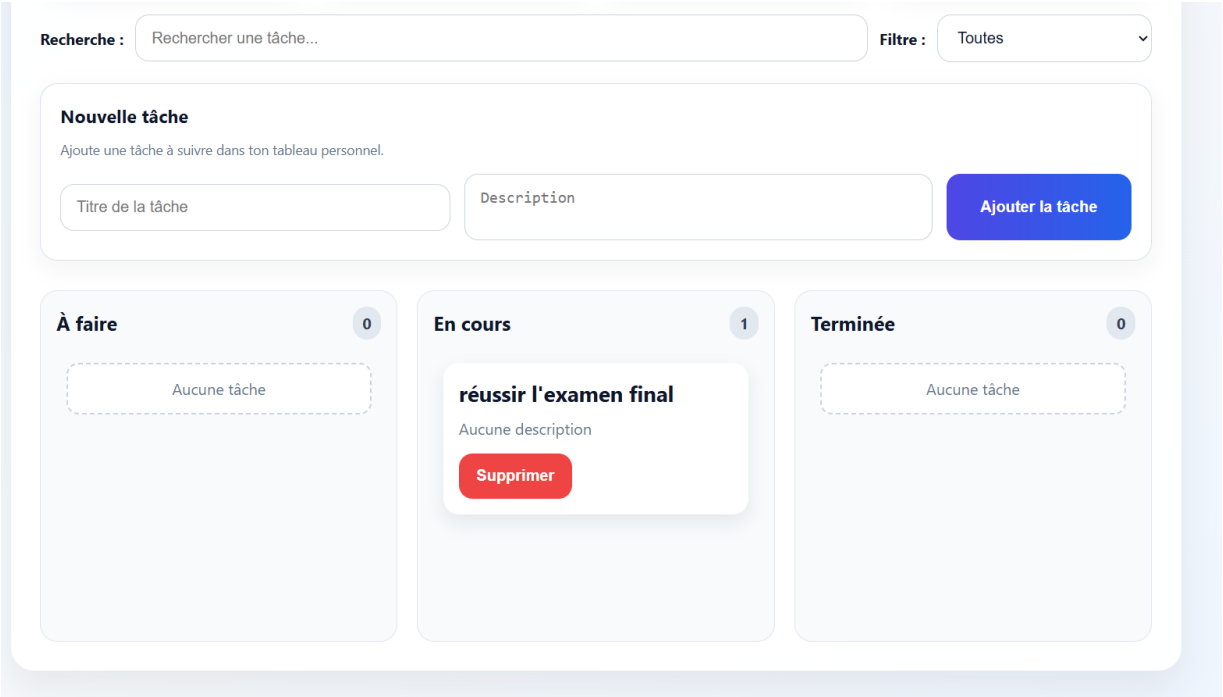
- **Consultation des tâches(Read)** :
Les tâches sont affichées sous forme de tableau Kanban avec trois colonnes : à faire, en cours et terminée.

- **Modification du statut (Update):**
L'utilisateur peut déplacer une tâche d'une colonne à une autre grâce au glisser-déposer, ce qui met à jour son statut.

- **Suppression de tâche (Delete):**
Chaque tâche peut être supprimée à l'aide d'un bouton dédié.

- Recherche et filtrage :
L'utilisateur peut rechercher une tâche ou filtrer les tâches selon leur statut.

Ces fonctionnalités permettent une gestion simple, rapide et intuitive des tâches..


Recherche :
Rechercher une tâche...
Filtre :
Toutes
Nouvelle tâche
Ajoute une tâche à suivre dans ton tableau personnel.
Titre de la tâche
Description
Ajouter la tâche
À faire
0
Aucune tâche
En cours
1
réussir l'examen final
Aucune description
Supprimer
Terminée
0
Aucune tâche

9 — Développement de la partie back-end

La partie **back-end** a été développée avec **Node.js** et **Express**. Elle permet de gérer l'authentification des utilisateurs ainsi que les opérations **CRUD** liées aux tâches.

9.1 — Choix réalisés

Le **back-end** de l'application a été développé avec Node.js et Express.

Le choix de **Node.js** permet d'utiliser **JavaScript côté serveur**, assurant une cohérence avec le **front-end** développé en **React**.

Le **framework Express** a été retenu pour sa simplicité et sa rapidité de mise en œuvre dans la création **d'API REST**.

L'architecture du **back-end** repose sur une séparation claire des responsabilités :
- **server.js** : point d'entrée de l'application
- **routes** : gestion des routes **API (authentification et tâches**)
- **middleware** : sécurisation des accès via **JWT**
- **database** : connexion à la base de données **MySQL**

La base de données **MySQL** a été choisie pour sa robustesse et sa compatibilité avec les applications web.

L'authentification repose sur l'utilisation de **JSON Web Token (JWT),** permettant de sécuriser les échanges entre le client et le serveur.

Les mots de passe sont chiffrés avec **bcrypt** avant d'être stockés, garantissant une protection des données sensibles.

Cette organisation permet d'obtenir une application sécurisée, maintenable et évolutive.

9.2 — Extrait de composant métier

```
1   const jwt = require("jsonwebtoken");
2
3   // Middleware qui protège les routes nécessitant une authentification
4   const authMiddleware = (req, res, next) => {
5       const authHeader = req.headers.authorization;
6
7       // Vérifie que le header Authorization existe
8       if (!authHeader || !authHeader.startsWith("Bearer ")) {
9           return res.status(401).json({ message: "Accès refusé" });
10      }
11
12      const token = authHeader.split(" ")[1];
13
14      try {
15          // Vérifie et décode le token JWT
16          const decoded = jwt.verify(token, process.env.JWT_SECRET);
17
18          // Ajoute les informations utilisateur dans la requête
19          req.user = decoded;
20
21          next();
22      } catch (error) {
23          res.status(401).json({ message: "Token invalide" });
24      }
25  };
26
27  module.exports = authMiddleware;
```

Middleware d'authentification JWT sécurisant l'accès aux routes de l'application.

L'extrait présenté correspond au middleware d'authentification utilisé dans le **back-end** de l'application.

Ce composant permet de sécuriser les routes en vérifiant la présence d'un **token JWT** dans **les requêtes HTTP**.

Le **token** est récupéré dans les en-têtes de la requête, puis vérifié à l'aide de la fonction **jwt.verify.**

Si le **token** est valide, les informations de l'utilisateur sont décodées et ajoutées à la requête **(req.user),** ce qui permet d'identifier l'utilisateur connecté.

En cas d'**absence** ou d'**invalidité** du **token**, l'accès est refusé avec une réponse **HTTP 401.**

Ce **middleware** est utilisé pour protéger toutes les routes liées aux tâches, garantissant que chaque utilisateur ne peut accéder qu'à ses propres données.

9.3 .— Extrait d'accès aux données

```
// Récupère toutes les tâches de l'utilisateur connecté
router.get("/", (req, res) => {
    db.query(
        "SELECT * FROM tasks WHERE user_id = ? ORDER BY created_at DESC",
        [req.user.id],
        (err, results) => {
            if (err) {
                return res.status(500).json({ message: "Erreur serveur" });
            }
            res.json(results);
        }
    );
});
```

Cet extrait montre un accès aux données réalisé dans le back-end de l'application.

Cet extrait correspond à une **route API** permettant de récupérer les tâches de l'utilisateur connecté.

La requête **SQL SELECT** interroge la **table tasks** en filtrant les résultats grâce à l'identifiant utilisateur contenu dans **req.user.id**.

Ce filtrage garantit qu'un **utilisateur** ne peut accéder qu'à **ses propres tâches**, assurant ainsi la **sécurité des données**.

Les résultats sont triés par **date de création** afin d'afficher les tâches les plus récentes en premier.

En **cas d'erreur,** le serveur renvoie une réponse **HTTP 500**. Sinon, les données sont renvoyées au **format JSON** vers le **front-end**.

Cet extrait illustre le lien entre la **logique métier**, **la sécurité** et **l'accès à la base de données**.

10 — Base de données : MCD, MLD et création

L'application repose sur une base de données relationnelle MySQL.

Deux entités principales sont utilisées :

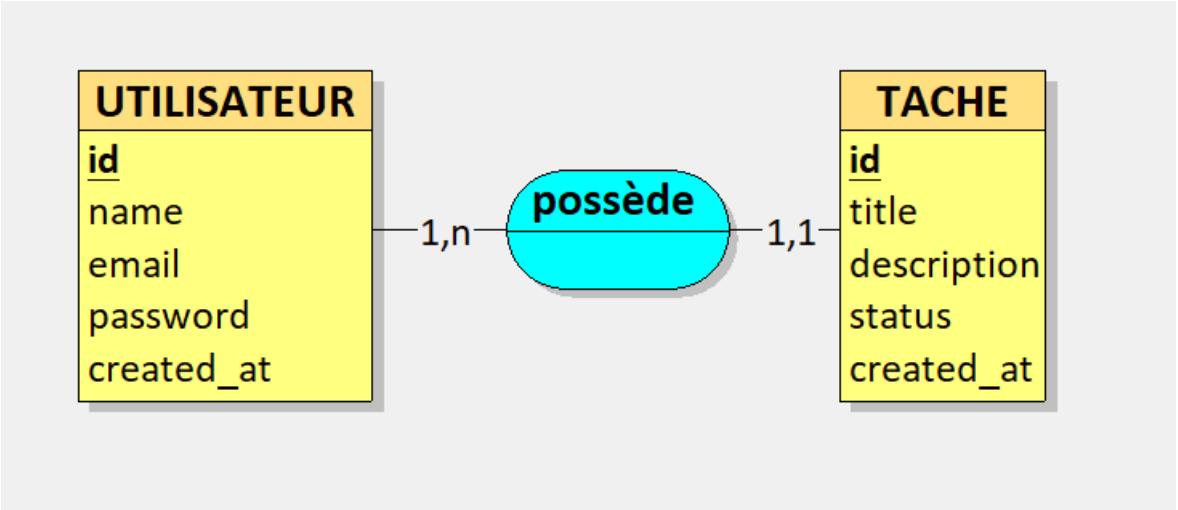
- **utilisateur ;** -

tâche.

Chaque tâche appartient à un utilisateur. On a donc une relation de type un-à-plusieurs : un utilisateur peut posséder plusieurs tâches.

10.1 — MCD

Le modèle conceptuel de données peut être résumé comme suit :

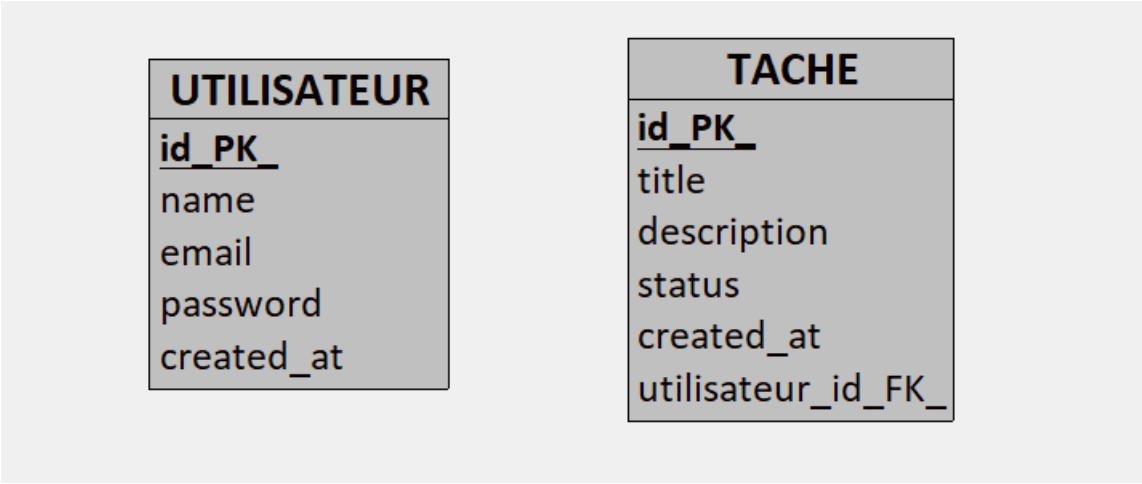


Modèle Conceptuel de Données (MCD) de l'application TaskPilot

Le modèle conceptuel de données met en évidence deux entités principales : **UTILISATEUR** et **TACHE**. Un utilisateur peut posséder plusieurs tâches, tandis qu'une tâche appartient à un seul utilisateur. La relation entre ces deux entités est donc de type 1 à N. Cette modélisation correspond au fonctionnement de l'application, dans laquelle chaque utilisateur gère uniquement ses propres tâches.

10.2 — MLD

Le modèle logique de données est le suivant :



Modèle Logique de Données (MLD) de l'application TaskPilot

Le modèle logique de données (MLD) représente la structure de la base de données relationnelle utilisée dans l'application.

Il se compose de deux tables principales :

- **USERS** : contient les informations des utilisateurs (identifiant, nom, email, mot de passe, date de création).
- **TASKS** : contient les tâches associées aux utilisateurs.

La **clé primaire (PK)** identifie de manière unique chaque enregistrement dans une table.

La **clé étrangère (FK) user_id** permet de relier chaque tâche à un utilisateur. Cette relation garantit l'intégrité des données et permet de filtrer les tâches en fonction de l'utilisateur connecté.

Ce modèle correspond directement à l'implémentation réalisée dans la base de données MySQL.

10.3 — Création SQL

Table users : CREATE TABLE users (id INT

AUTO_INCREMENT PRIMARY KEY, name

VARCHAR(255) NOT NULL, email

VARCHAR(255) UNIQUE NOT NULL,

password VARCHAR(255) NOT NULL,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

Table tasks : CREATE TABLE tasks (id INT

AUTO_INCREMENT PRIMARY KEY, title

VARCHAR(255) NOT NULL, description

TEXT, status VARCHAR(50) DEFAULT

'todo', user_id INT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

);

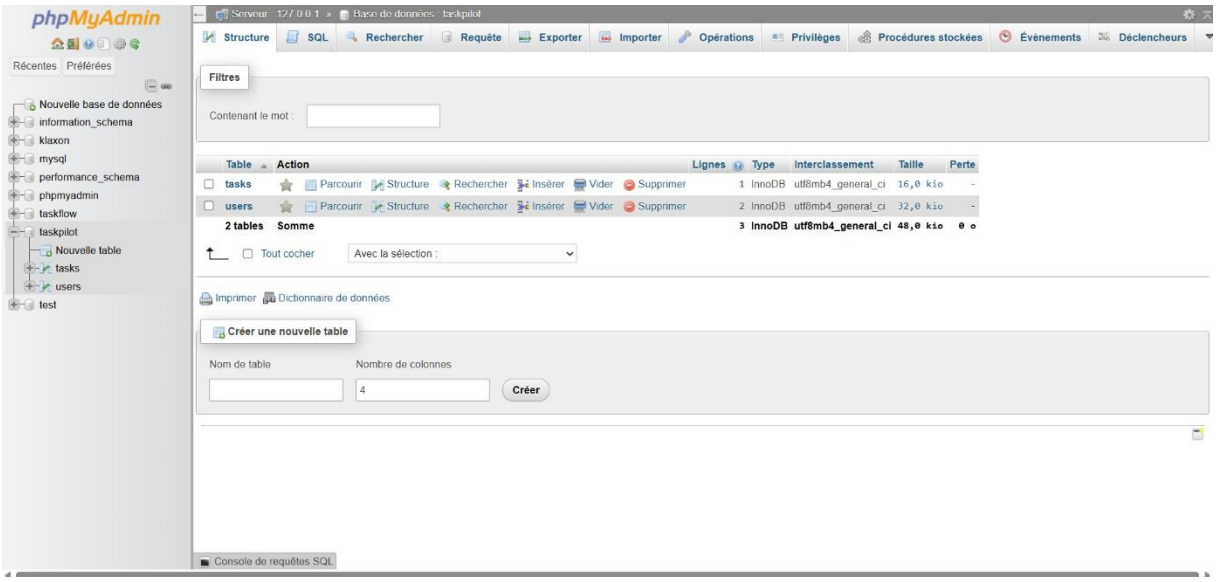
Les tables de la base de données ont été créées à l'aide de requêtes **SQL** exécutées dans **MySQL**.

La **table users** permet de stocker les informations des utilisateurs, notamment **leur nom**, **leur email** et **leur mot de passe chiffré**.

La **table tasks** contient les tâches associées aux utilisateurs. Chaque tâche possède un identifiant utilisateur (**user_id**) qui permet de faire le lien avec la **table users.**

Une contrainte de **clé étrangère (FOREIGN KEY**) a été mise en place afin d'assurer l'intégrité des données. Ainsi, lorsqu'un utilisateur est supprimé, toutes ses tâches associées sont également supprimées automatiquement.

Ces **requêtes SQL** permettent de structurer efficacement les données de l'application.



11 — Sécurité de l'application

La sécurité constitue un point important du projet TaskPilot.

Plusieurs mécanismes ont été mis en place :

- chiffrement des mots de passe avec bcrypt ;

- authentification par token JWT ; - middleware de protection des routes ;

- filtrage des tâches par utilisateur connecté.

Lors de l'inscription, le **mot de passe est hashé** avant d'être enregistré en base de données. Lors de la connexion, le mot de passe saisi est comparé au hash enregistré.

Après authentification, un **token JWT** est généré puis renvoyé au client. Ce **token** est ensuite transmis dans les requêtes afin d'accéder aux ressources protégées.

Chaque requête liée aux tâches repose sur l'identifiant utilisateur extrait du **token**, ce qui empêche un utilisateur d'accéder aux tâches d'un autre.

Une validation des données est effectuée côté serveur afin d'éviter les erreurs et les injections.

Les **requêtes SQL** utilisent des paramètres **(req.user.id)** afin d'éviter les **injections SQL**.

Les routes sont protégées par un **middleware JWT** qui vérifie systématiquement l'identité de l'utilisateur.

Une vérification est également effectuée côté front-end pour empêcher l'accès aux pages sans authentification.

11.1 — Présentation des éléments de sécurité

```
8   // Route d'inscription
9   router.post("/register", async (req, res) => {
10      const { name, email, password } = req.body;
11
12      // Vérifie que tous les champs sont remplis
13      if (!name || !email || !password) {
14          return res.status(400).json({ message: "Tous les champs sont obligatoires" });
15      }
16
17      try {
18          // Hash du mot de passe avant enregistrement
19          const hashedPassword = await bcrypt.hash(password, 10);
20
21      db.query(
22          "INSERT INTO users (name, email, password) VALUES (?, ?, ?)",
23          [name, email, hashedPassword],
24          (err) => {
25              if (err) {
26                  return res.status(500).json({ message: "Erreur serveur", error: err });
27              }
28
29              res.status(201).json({ message: "Utilisateur créé" });
30          }
31      );
32      } catch (error) {
33          res.status(500).json({ message: "Erreur serveur" });
34      }
35  });
```

```
7
8   // Ajoute automatiquement le token JWT dans les requêtes protégées
9   api.interceptors.request.use((config) => {
10      const token = localStorage.getItem("token");
11
12      if (token) {
13          config.headers.Authorization = `Bearer ${token}`;
14      }
15
16      return config;
17  });
```

La sécurité de l'application repose sur plusieurs mécanismes complémentaires.

Tout d'abord, les mots de passe des utilisateurs sont chiffrés à l'aide de la bibliothèque bcrypt avant d'être stockés en base de données. Cela permet d'éviter toute récupération des mots de passe en clair.

Ensuite, un token JWT est généré lors de la connexion. Ce token est utilisé pour authentifier les requêtes envoyées au serveur. Il est transmis dans les headers HTTP et vérifié via un middleware.

Enfin, les routes de gestion des tâches sont protégées. Chaque requête vérifie l'identité de l'utilisateur, ce qui empêche l'accès aux données d'un autre utilisateur.

11.2 — Jeu d'essai de sécurité

Le jeu d'essai de sécurité consiste notamment à vérifier :
- la connexion avec un mot de passe correct ;
- le refus de connexion avec un mot de passe incorrect ;
- le refus d'accès aux routes de tâches sans token ;
- l'impossibilité d'accéder aux données d'un autre utilisateur.

12 — Jeu d'essai

Afin de valider le fonctionnement de l'application, un jeu d'essai a été mis en place.

Les données de test utilisées sont :

- un ou plusieurs utilisateurs enregistrés ;
- plusieurs tâches avec différents statuts ;
- des cas d'erreur volontaires, comme une connexion avec identifiants incorrects ou l'absence de token.

Cas testé	Données	Résultat attendu	Résultat obtenu
Connexion	Email + mot de passe corrects	Accès dashboard	OK
Connexion invalide	Mot de passe faux	refus	OK
Création tâche	Titre + description	Tâche créée	OK
Drag & Drop	Tâche déplacée	Statut mis à jour	OK
Suppression	Clic supprimer	Tâche supprimée	OK

13 — Tests réalisés

Des tests fonctionnels ont été réalisés sur l'ensemble de l'application.

Les tests ont porté sur :

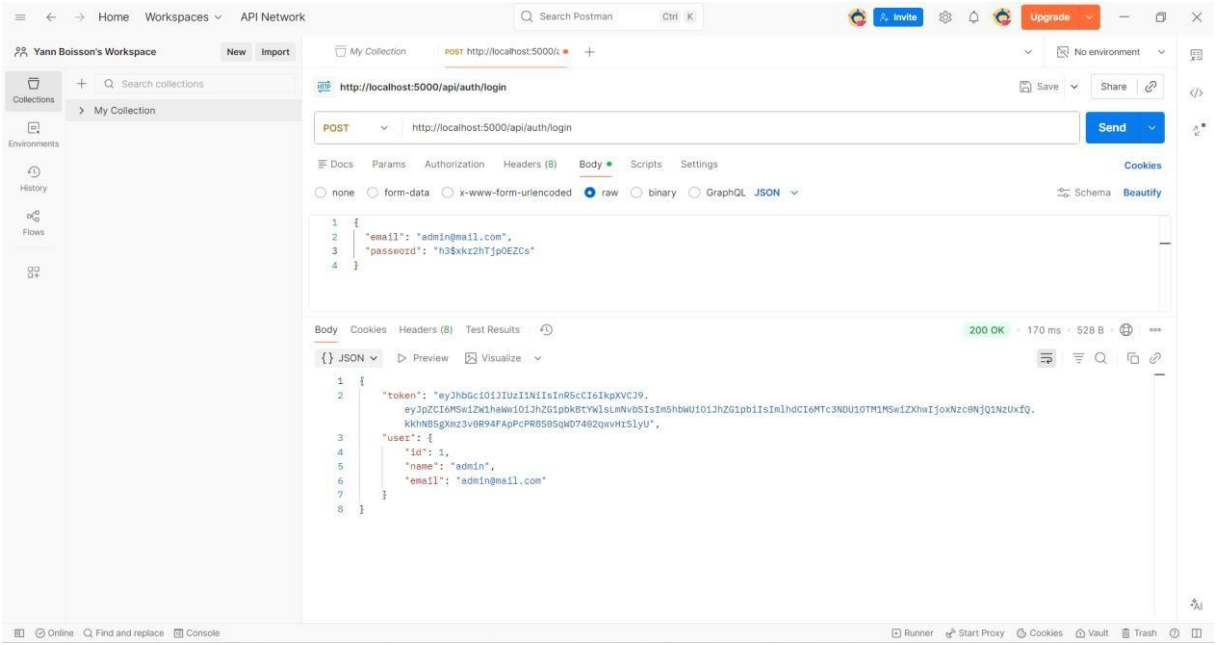
- **l'inscription** utilisateur ;
- la **connexion** ;
- **l'accès** protégé aux routes ;

- **la création** d'une tâche ;

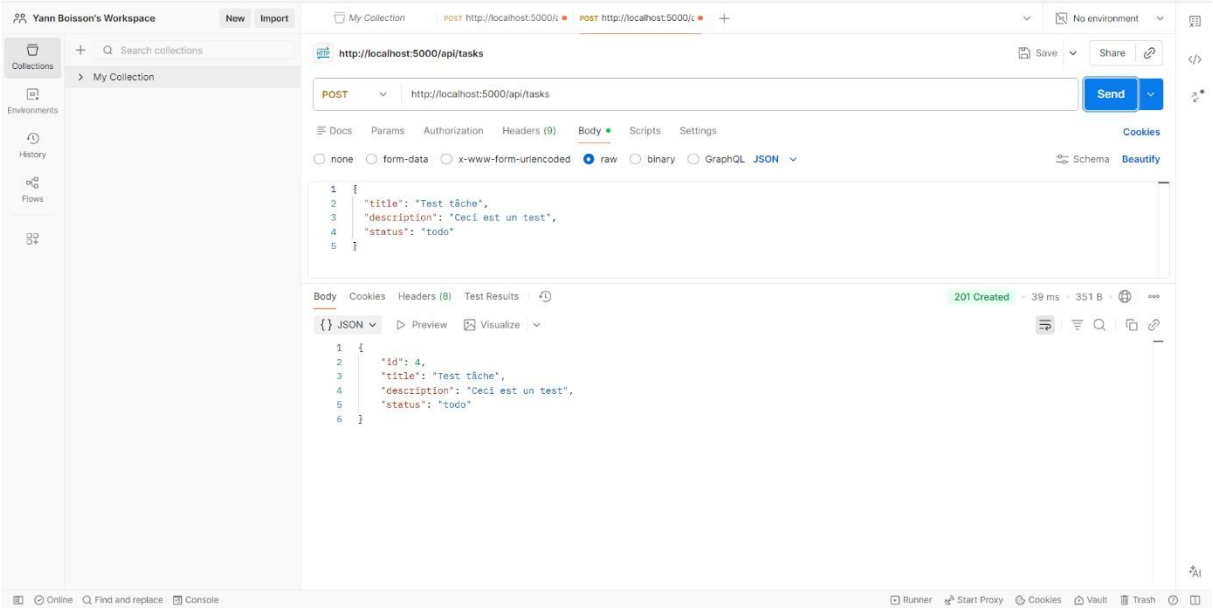
- **la suppression** d'une tâche ;

- **le déplacement** d'une tâche entre colonnes ; **- la recherche** et **le filtrage**.

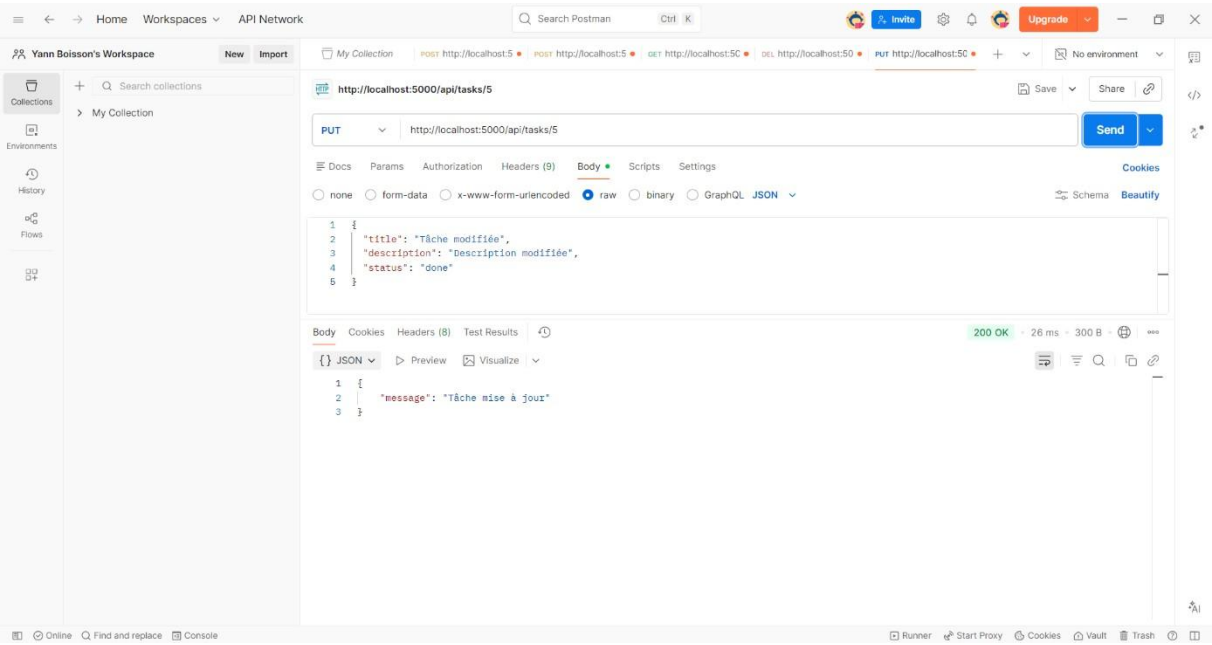
Ces tests ont été réalisés manuellement en environnement local, complétés par des vérifications via les outils de développement du navigateur et des appels API.



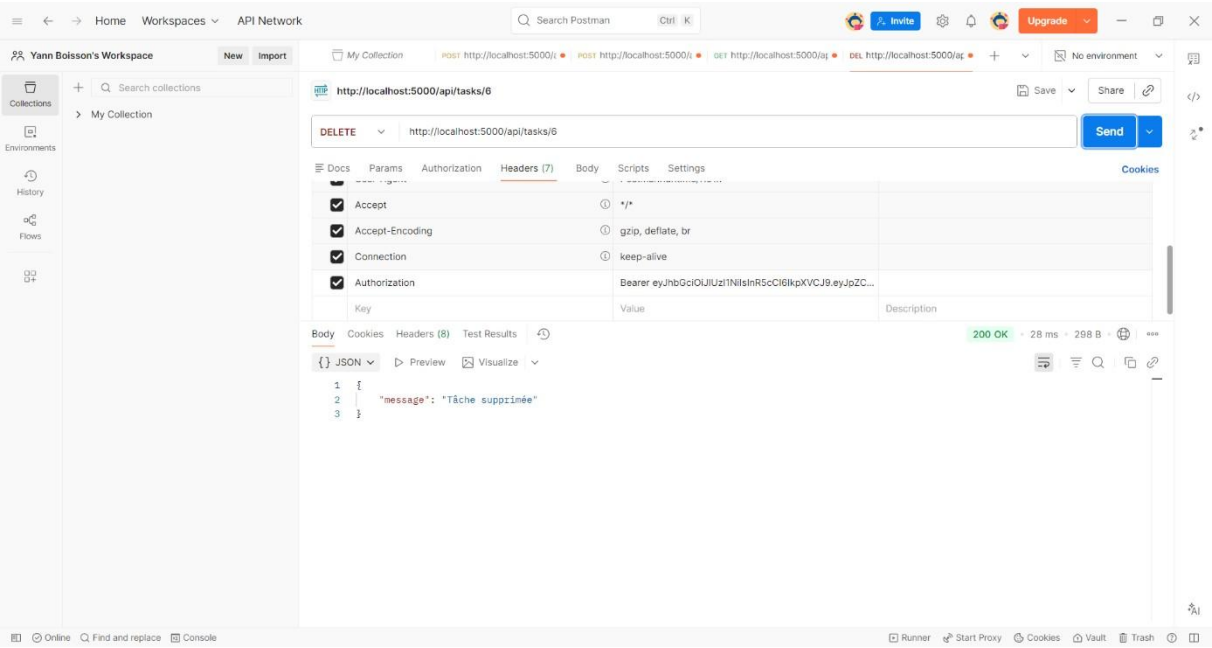
Test de connexion via Postman avec récupération du token JWT



Test de création d'une tâche via l'API avec authentification JWT



Déplacement d'une tâche entre colonnes



Suppression d'une tâche via API

Les tests ont été réalisés à l'aide de **Postman** et du **navigateur** afin de valider chaque fonctionnalité.

Chaque fonctionnalité **CRUD** a été testée individuellement afin de vérifier son bon fonctionnement.

Les captures présentées permettent de prouver que l'application fonctionne correctement dans des conditions réelles.

—

14 Veille sur les vulnérabilités de sécurité

Une veille simplifiée a été réalisée sur les vulnérabilités les plus courantes des applications web.

Les principaux points de vigilance identifiés sont :

- **vol** ou exposition des tokens JWT ;
- **faiblesse** des mots de passe ;
- **injection** SQL ;
- **mauvaise** gestion des droits d'accès ;
- **exposition** de données sensibles dans le code source ou dans le dépôt Git.

Dans le cadre du projet, plusieurs bonnes pratiques ont été retenues :
- non-versionnage du fichier .env ;
- utilisation de requêtes paramétrées ;
- hash des mots de passe avec bcrypt ;
- protection des routes avec middleware JWT ;
- séparation des responsabilités entre front-end et back-end.

15 Conclusion

Le projet **TaskPilot** a permis de concevoir une **application web full-stack** complète répondant à un besoin concret de gestion de tâches.

Il a mobilisé des compétences en développement **front-end** avec **React**, en développement **back-end** avec **Node.js / Express**, en gestion de **base de données MySQL** et en **sécurisation d'application avec JWT** et **bcrypt**.

La mise en œuvre d'un **tableau Kanban** interactif, d'une authentification sécurisée et d'une architecture séparant front-end et back-end constitue un point fort du projet.

Ce projet m'a permis de consolider mes compétences techniques, d'améliorer ma capacité à structurer une application web et de mieux comprendre les attentes d'un projet professionnel complet.

—

16 Annexes

Cette section regroupe des éléments complémentaires permettant d'appuyer la compréhension technique du projet TaskPilot. Ces éléments ne sont pas indispensables à la lecture principale du dossier, mais apportent des preuves concrètes du travail réalisé.

16.1 — Arborescence du projet

L'arborescence suivante présente l'organisation globale du projet :

```
taskpilot/
|
 ├── backend/
| ├── config/
| ├── middleware/
| ├── routes/
| ├── database/
| └── server.js
|
└── frontend/
 ├── components/
 ├── pages/
 ├── services/
└── App.jsx
```

Cette organisation permet de séparer clairement les responsabilités entre le front-end, le back-end et la gestion des données.

16.2 — README du projet

Le fichier README présente le projet, ses fonctionnalités, les technologies utilisées ainsi que les étapes d'installation et de lancement.

 # TaskPilot

Application web full-stack de gestion de tâches avec authentification sécurisée et tableau Kanban interactif en drag & drop.

—

---## Présentation

TaskPilot permet à un utilisateur de gérer ses tâches quotidiennes dans une interface moderne et intuitive.

Chaque utilisateur possède un espace personnel sécurisé pour :

- créer des tâches
- modifier des tâches
- supprimer des tâches
- organiser les tâches par statut

Fonctionnalités

- Authentification sécurisée avec JWT
- CRUD complet des tâches
- Tableau Kanban avec drag & drop (dnd-kit)
- Recherche dynamique
- Filtrage par statut
- Dashboard avec statistiques
- Interface responsive

Technologies

Frontend

- React
- React Router
- Axios
- dnd-kit

- CSS responsive

Backend

- Node.js - Express
- MySQL
- JWT
- bcrypt

Installation

1. Cloner le projet

git clone https://github.com/yannb13/taskpilot.git cd taskpilot

2. Configurer le backend

Créer un fichier `.env` dans le dossier backend :

PORT=5000

DB_HOST=localhost

DB_USER=root

DB_PASSWORD=

DB_NAME=taskpilot

JWT_SECRET=my_secret_key

Installer et lancer :

```
cd backend
npm install
npm run dev
```

3. Lancer le frontend

```
cd frontend
npm install
npm run dev
```

Application accessible sur :

http://localhost:5173

Base de données

Créer une base MySQL :

```
taskpilot
```

Puis exécuter :

```
CREATE TABLE tasks ( id INT
AUTO_INCREMENT PRIMARY KEY, title
VARCHAR(255) NOT NULL, description
TEXT,
```

status VARCHAR(50) DEFAULT 'todo', user_id

INT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

Structure

taskpilot/ backend/

frontend/

Sécurité

- Authentification JWT
- Hash des mots de passe avec bcrypt
- Protection des routes API
- Données isolées par utilisateur

Améliorations possibles

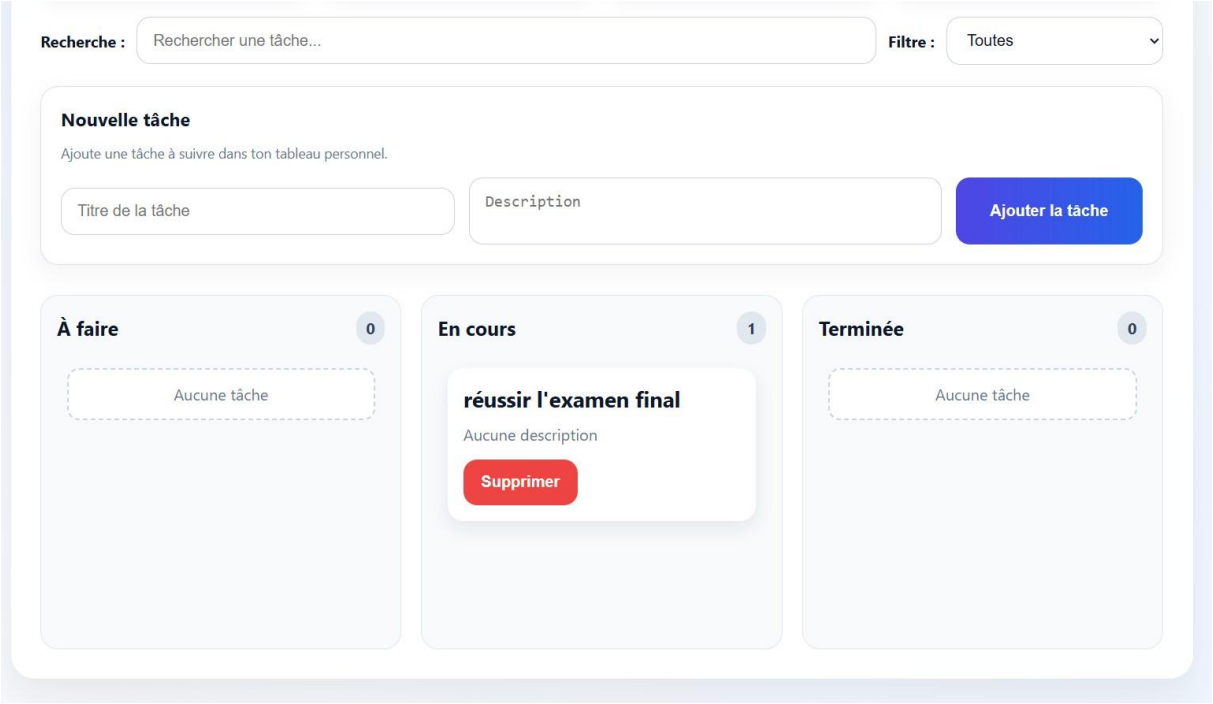
- Mode sombre
- Priorité des tâches
- Dates d’échéance
- Notifications temps réel
- Version mobile optimisée
- Déploiement cloud

Auteur

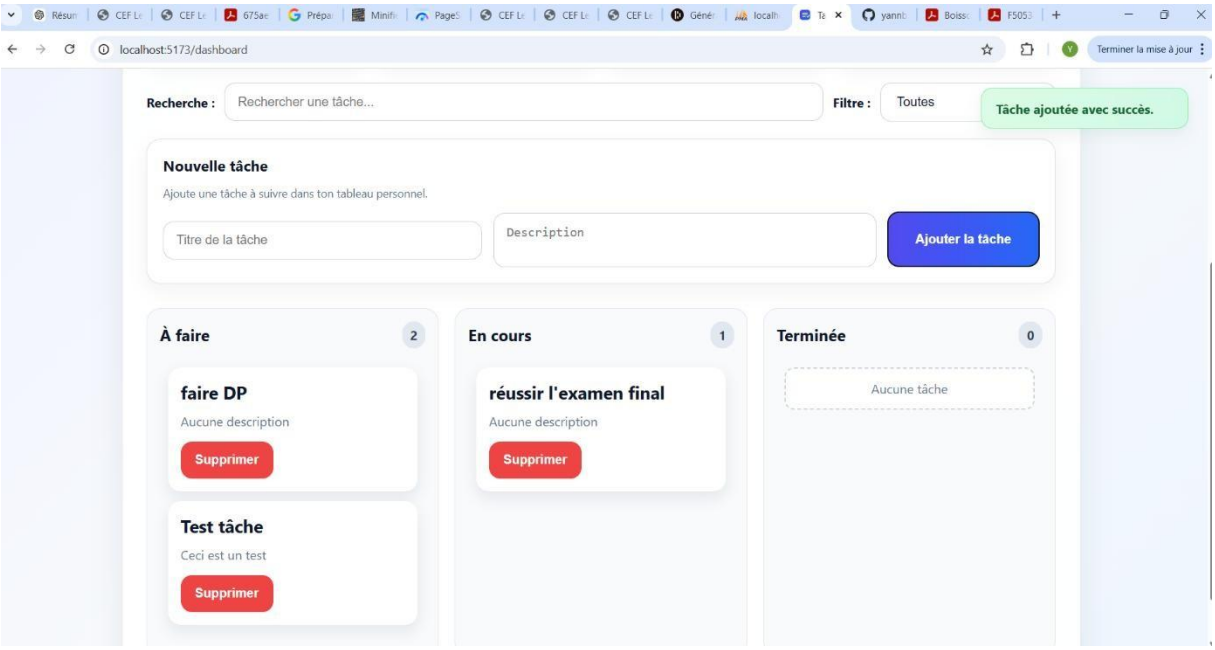
Yann BOISSON

Formation Développeur Web et Web Mobile

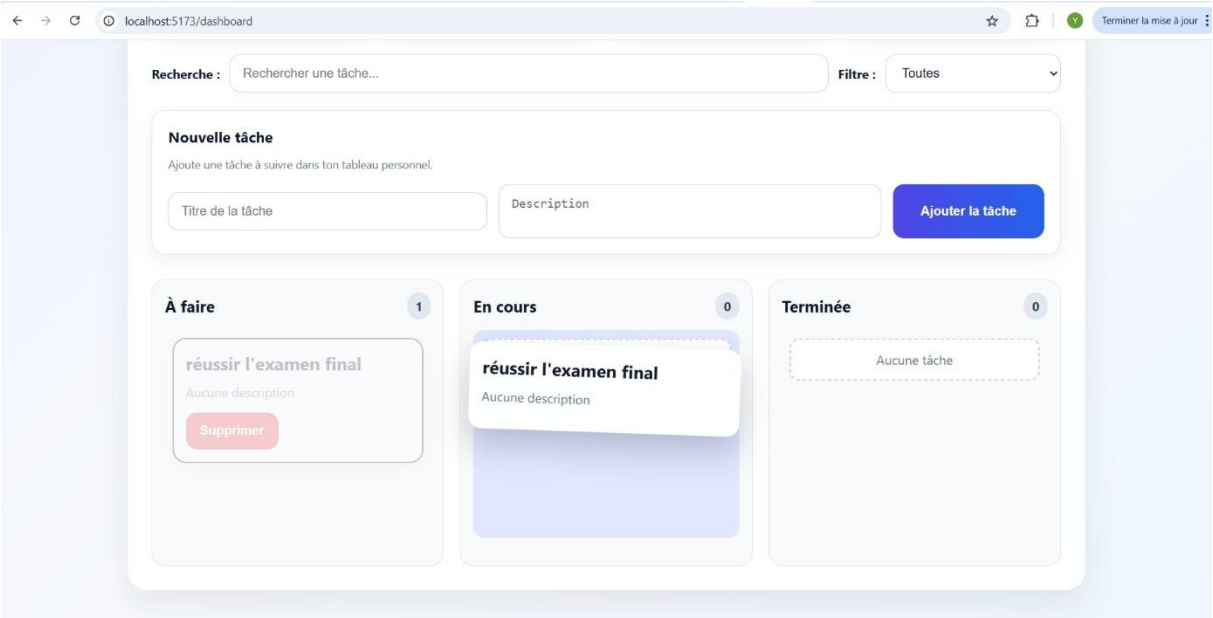
16.3 — Captures supplémentaires



Interface du tableau Kanban



Création d'une tâche



Déplacement d'une tâche (drag & drop)

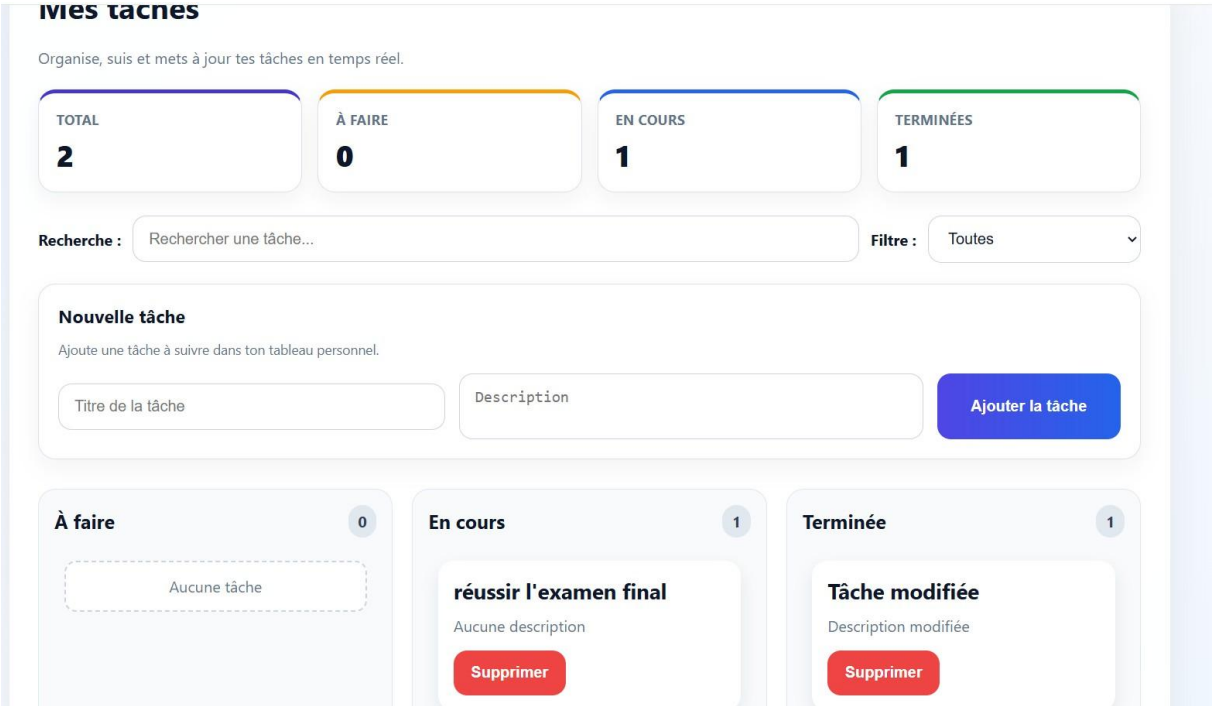
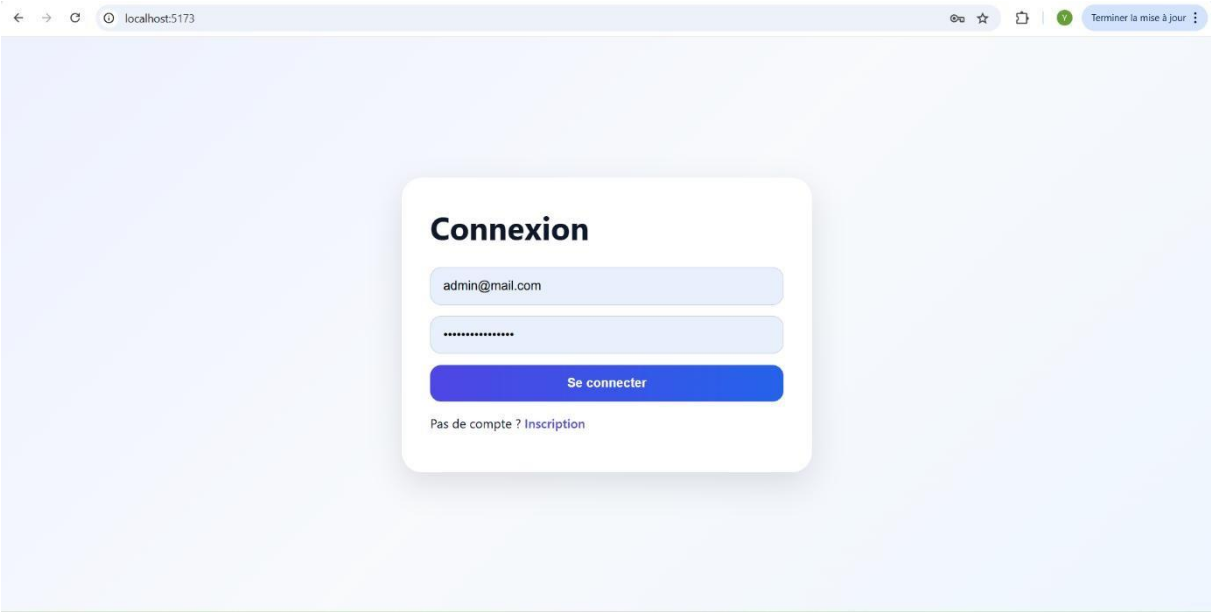


Tableau de bord avec statistiques



Connexion utilisateur

16.4 — Extraits de code complémentaires

Les extraits suivants permettent d'illustrer certains aspects techniques importants du projet.

```
taskflow > backend > middleware > JS authMiddleware.js > ...
1   const jwt = require("jsonwebtoken");
2
3   // Middleware qui protège les routes nécessitant une authentification
4   const authMiddleware = (req, res, next) => {
5       const authHeader = req.headers.authorization;
6
7       // Vérifie que le header Authorization existe
8       if (!authHeader || !authHeader.startsWith("Bearer ")) {
9           return res.status(401).json({ message: "Accès refusé" });
10      }
11
12      const token = authHeader.split(" ")[1];
13
14      try {
15          // Vérifie et décode le token JWT
16          const decoded = jwt.verify(token, process.env.JWT_SECRET);
17
18          // Ajoute les informations utilisateur dans la requête
19          req.user = decoded;
20
21          next();
22      } catch (error) {
23          res.status(401).json({ message: "Token invalide" });
24      }
25  };
26
27  module.exports = authMiddleware;
```

Cet extrait montre comment l'accès aux routes est sécurisé via un token JWT.

```
// Récupère toutes les tâches de l'utilisateur connecté
router.get("/", (req, res) => {
    db.query(
        "SELECT * FROM tasks WHERE user_id = ? ORDER BY created_at DESC",
        [req.user.id],
        (err, results) => {
            if (err) {
                return res.status(500).json({ message: "Erreur serveur" });
            }
            res.json(results);
        }
    );
});
```

Cet extrait permet de récupérer toutes les tâches de l'utilisateur connecté en filtrant par user_id.

```
// Crée une nouvelle tâche
router.post("/", (req, res) => {
    const { title, description, status } = req.body;

    if (!title) {
        return res.status(400).json({ message: "Le titre est obligatoire" });
    }

    db.query(
        "INSERT INTO tasks (title, description, status, user_id) VALUES (?, ?, ?, ?)",
        [title, description || "", status || "todo", req.user.id],
        (err, result) => {
            if (err) {
                return res.status(500).json({ message: "Erreur serveur" });
            }

            res.status(201).json({
                id: result.insertId,
                title,
                description,
                status: status || "todo",
            });
        }
    );
});
```

Cet extrait permet de créer une nouvelle tâche associée à l'utilisateur connecté.

```
// Supprime une tâche
router.delete("/:id", (req, res) => {
    const taskId = req.params.id;

    db.query(
        "DELETE FROM tasks WHERE id = ? AND user_id = ?",
        [taskId, req.user.id],
        (err) => {
            if (err) {
                return res.status(500).json({ message: "Erreur serveur" });
            }

            res.json({ message: "Tâche supprimée" });
        }
    );
});
```

Cet extrait permet de supprimer une tâche en vérifiant qu'elle appartient bien à l'utilisateur connecté.

16.5 — Requêtes SQL utilisées

Création de la table users :

```
CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100), email
VARCHAR(255) UNIQUE, password
VARCHAR(255),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Création de la table tasks :

```
CREATE TABLE tasks ( id INT
AUTO_INCREMENT PRIMARY KEY, title
VARCHAR(255) NOT NULL, description
TEXT,
status VARCHAR(50) DEFAULT 'todo',
user_id INT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Ces tables permettent de gérer les utilisateurs ainsi que leurs tâches associées.